Data Structures & Algorithms

GOLDEN GATE UNIVERSITY

Session 5 – Stacks and Their Applications

Dr. Yatish Bathla,

Ph.D. (Information Technology), Obuda University, Budapest

What you will be able to do today

Explain the LIFO rule and the stack abstract data type

Trace push, pop and peek on a diagram

Implement a stack with an array and with a linked list

Handle overflow and underflow

Apply stacks: reversing, bracket matching, undo and postfix

Stacks in everyday life

Plates

Cafeteria

Add a plate on top; take the top plate first.

Undo

Text editor

Undo reverses the most recent edit first.

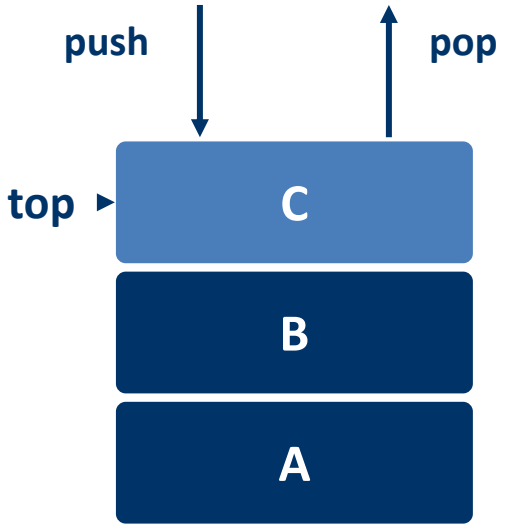
Back

Web browser

Back returns to the last page you visited.

The most recent item is handled first: Last In, First Out (LIFO).

LIFO: last in, first out



The rule

push: place an item on the top

pop: remove the item on the top

Only the top item is visible

C went in last, so C comes out first

A, B, C go in; C, B, A come out.

The stack ADT: operations

Operation	What it does	Cost
push(x)	Put x on the top	O(1)
pop()	Remove and return the top item	O(1)
peek()	Look at the top item, do not remove	O(1)
isEmpty()	True when there are no items	O(1)
size()	Number of items stored	O(1)

Lafore says peek, Goodrich says top: same operation.

Tracing a stack

Operation	Stack (bottom to top)	Returned
push(5)	5	-
push(8)	5, 8	-
push(3)	5, 8, 3	-
pop()	5, 8	3
peek()	5, 8	8
pop()	5	8

peek reads the top; pop reads and removes it.

Why restrict access?

A stack allows

Add on top (push)

Remove from top (pop)

Read the top (peek)

A stack does not allow

Reading the middle

Removing from the middle

Searching by position

Restricted access makes a stack simple, safe and fast.

Array-based stack

```
class ArrayStack {
    private int[] data;
    private int top = -1;   // -1 means empty

    ArrayStack(int cap) { data = new int[cap]; }

    void push(int x) { data[++top] = x; }
    int pop()         { return data[top--]; }
    int peek()        { return data[top]; }
    boolean isEmpty() { return top == -1; }
    boolean isFull()  { return top == data.length - 1; }
}
```

Key ideas

top is the index of the top item

push: raise top, then store

pop: read, then lower top

Capacity is fixed: isFull() matters

Overflow and underflow

Overflow

Push on a full stack

Array is full: top == length - 1. Check isFull() or grow the array.

Underflow

Pop an empty stack

Nothing to remove: top == -1. Check isEmpty() or throw an exception.

Resize

The dynamic fix

Make a bigger array, copy items, carry on. Linked stacks never fill up.

Test isEmpty() before every pop() or peek().

Linked-list stack

```
class LinkedStack {
    private Node head;          // head is the top

    void push(int x) {
        Node n = new Node(x);
        n.next = head;
        head = n;
    }
    int pop() {
        if (head == null)
            throw new RuntimeException("Stack empty");
        int x = head.data;
        head = head.next;
        return x;
    }
    boolean isEmpty() { return head == null; }
}
```

Key ideas

push is insertFirst

pop is deleteFirst

Reuses last session's list

No overflow: grows with memory

Array stack vs linked stack

Feature	Array stack	Linked stack
Capacity	Fixed (or resize)	Grows as needed
push / pop time	O(1)	O(1)
Memory	One block, no extra references	One extra reference per node
Overflow	Possible	Only if memory runs out
Use when	Maximum size is known	Size is unknown

Same ADT, two implementations: the user code does not change.

Efficiency of stacks

O(1)

push

One index or pointer change.

O(1)

pop

Remove the top only.

O(1)

peek

Read the top only.

O(n)

Space

One cell or node per item.

Cost does not depend on stack size.

Activity: think, pair, share

Problem

Start with an empty stack. Run: push(4), push(7), pop(), push(1), push(9), pop(), pop(). What is left, and which values were popped, in order?

Application 1: reversing a word

```
static String reverse(String s) {
    Deque<Character> stack = new ArrayDeque<>();
    for (char c : s.toCharArray())
        stack.push(c);            // push letters
    StringBuilder sb = new StringBuilder();
    while (!stack.isEmpty())
        sb.append(stack.pop());  // pop letters
    return sb.toString();
}
```

Key ideas

Push S, T, A, C, K

Pop K, C, A, T, S

LIFO reverses the order for free

O(n) time and O(n) space

Java: prefer ArrayDeque to the old Stack class

Application 2: matching brackets

Expression	Balanced?	Reason
(a + b) * c	Yes	Every opener is closed in order
[(a + b) * c]	Yes	Inner pair closes before the outer
(a + b]	No	] does not match the open (
((a + b)	No	One opener is never closed
a + b)	No	A closer arrives on an empty stack

Compilers and editors run this check every time you save code.

Bracket matching: the algorithm

```
static boolean balanced(String s) {
    Deque<Character> st = new ArrayDeque<>();
    for (char c : s.toCharArray()) {
        if (c == '(' || c == '[' || c == '{') {
            st.push(c);
        } else if (c == ')' || c == ']' || c == '}') {
            if (st.isEmpty()) return false;
            char open = st.pop();
            if (!matches(open, c)) return false;
        }
    }
    return st.isEmpty();
}
```

Key ideas

Opener: push it

Closer: pop and compare

Mismatch or empty stack: not balanced

At the end the stack must be empty

Cost: O(n)

Application 3: undo, back and calls

Undo

Editors

Each edit is pushed; Undo pops the latest one.

Back

Browsers

Visited pages are pushed; Back pops to the previous page.

Calls

Call stack

Each method call is pushed; return pops it. Recursion relies on this.

Whenever you must backtrack in reverse order, think stack.

Application 4: postfix expressions

Token	Action	Stack (bottom to top)
3	push 3	3
4	push 4	3, 4
+	pop 4 and 3, push 3 + 4	7
2	push 2	7, 2
*****	pop 2 and 7, push 7 * 2	14

Infix: (3 + 4) * 2 Postfix: 3 4 + 2 * Result: 14

Postfix needs no brackets: the stack remembers the order.

Mini quiz

1. Items 1, 2, 3 are pushed. In what order do they pop?
2. Which operation reads the top without removing it?
3. Why check isEmpty() before pop()?
4. Which list operations do push and pop use in a linked stack?
5. Are {[()]} and ([)] balanced?

Recap and check for understanding

A stack is LIFO: push and pop at one end, the top

All operations are O(1); space is O(n)

Array stack: fixed capacity. Linked stack: grows as needed

Guard against overflow and underflow

Applications: reversing, bracket matching, undo, call stack, postfix

Next session: Java lab building lists and stacks